

CIA 2 – Part A

Association Rule Mining: Apriori and FP-Growth

Student: Naveen Kumar

Programme: BBA (Finance & Marketing Analytics)

Dataset: ecommerce_dataset_updated.csv

This report implements and compares the Apriori and FP-Growth association-rule mining algorithms on an e-commerce categorical transaction dataset. The study focuses on frequent itemsets, support, confidence, lift, algorithmic approach and computational efficiency.

1. Objective

The objective is to identify recurring relationships among categorical purchase attributes and to compare two major frequent-pattern mining algorithms. Apriori uses candidate generation and pruning, whereas FP-Growth compresses the transaction database into an FP-tree and mines conditional pattern bases.

2. Dataset Description

The dataset contains **3,660 rows and 8 original columns**. The original fields include User_ID, Product_ID, Category, Price, Discount, Final Price, Payment Method and Purchase Date. User_ID and Product_ID are unique in this dataset, so using User_ID as a market basket would create one-item transactions. To make the dataset suitable for categorical association mining, each original purchase row is treated as one transaction containing five categorical items.

3. Preprocessing

Category and Payment Method are already categorical. Price, Discount and Final Price are numerical, so they are converted into Low, Medium and High groups using quantile-based binning. Quantile bins reduce the risk of highly imbalanced arbitrary cut-offs. Missing values were checked during execution; the selected fields required for transaction construction were available for all records.

Original field	Association item
Category	Category=<value>
Payment_Method	Payment=<value>
Price (Rs.)	Price=Low/Medium/High
Discount (%)	Discount=Low/Medium/High
Final_Price(Rs.)	FinalPrice=Low/Medium/High

4. Association Rule Measures

Support measures how often an itemset appears in all transactions. $\text{Support}(X \rightarrow Y) = \text{Support}(X \cup Y)$.

Confidence measures the conditional probability of Y given X. $\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$.

Lift compares the observed confidence with the expected occurrence of Y. $\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$. A lift above 1 indicates a positive association.

5. Parameters

Minimum support = **5%**; minimum confidence = **50%**. These thresholds balance the number of patterns with interpretability for this relatively small categorical item universe.

6. Apriori Implementation

The Apriori implementation starts with frequent 1-itemsets, generates larger candidate itemsets, counts their occurrence in the transactions and prunes candidates that do not meet minimum support. The key Apriori principle is that every subset of a frequent itemset must also be frequent. This pruning reduces unnecessary candidate evaluation, although candidate generation can become expensive as the number of items grows.

7. FP-Growth Implementation

The FP-Growth implementation first counts frequent individual items, builds an FP-tree and then recursively mines conditional pattern bases. Unlike Apriori, it does not repeatedly generate and test large candidate sets. The algorithm therefore avoids a major source of computational overhead associated with Apriori.

8. Frequent Itemset Results

Algorithm	Frequent itemsets	Rules	Execution time (reference run)
Apriori	126	49	0.104258 s
FP-Growth	126	49	0.064280 s

Both algorithms found **126** frequent itemsets, with a maximum itemset size of **3**. They also produced **49** rules at the selected confidence threshold. The verification step showed that the frequent itemsets and support values agree between the two implementations.

9. Sample Association Rules

Antecedent	Consequent	Support	Confidence	Lift
Discount=Discount_Low, Price=Price_High	FinalPrice=FinalPrice_High	0.133	1.000	3.002
Discount=Discount_Low, FinalPrice=FinalPrice_Low	Price=Price_Low	0.103	1.000	3.000
Discount=Discount_High, Price=Price_Low	FinalPrice=FinalPrice_Low	0.078	1.000	3.000
Discount=Discount_Medium, FinalPrice=FinalPrice_Low	Price=Price_Low	0.121	0.978	2.934
Discount=Discount_Medium, Price=Price_High	FinalPrice=FinalPrice_High	0.114	0.974	2.926
Discount=Discount_Medium, Price=Price_Low	FinalPrice=FinalPrice_Low	0.121	0.963	2.889
FinalPrice=FinalPrice_Medium, Price=Price_High	Discount=Discount_High	0.053	0.946	3.939
FinalPrice=FinalPrice_Low, Payment=Cash on Delivery	Price=Price_Low	0.059	0.943	2.830
Discount=Discount_Medium, FinalPrice=FinalPrice_Medium	Price=Price_Medium	0.114	0.937	2.811
Payment=UPI, Price=Price_Low	FinalPrice=FinalPrice_Low	0.063	0.920	2.760
FinalPrice=FinalPrice_Low, Payment=Net Banking	Price=Price_Low	0.057	0.913	2.739
Payment=Debit Card, Price=Price_Low	FinalPrice=FinalPrice_Low	0.059	0.912	2.735
FinalPrice=FinalPrice_Low, Payment=Debit Card	Price=Price_Low	0.059	0.912	2.735
Discount=Discount_Medium, FinalPrice=FinalPrice_High	Price=Price_High	0.114	0.909	2.727
Price=Price_Low	FinalPrice=FinalPrice_Low	0.302	0.906	2.717

The complete rule tables are supplied separately as CSV files in the outputs folder. Apriori and FP-Growth generate the same rules because they discover the same frequent itemsets under identical thresholds.

10. Comparison of Apriori and FP-Growth

Basic approach: Apriori repeatedly generates candidate itemsets and scans transactions to count support. FP-Growth builds a compressed FP-tree and mines conditional pattern bases.

Candidate generation: Apriori explicitly generates candidates. FP-Growth avoids the repeated candidate-generation process.

Database scans: Apriori may require multiple passes over the transaction database. FP-Growth uses an initial frequency count and then works mainly with FP-trees and conditional databases.

Memory: Apriori may store many candidate itemsets. FP-Growth stores a tree structure, which can be efficient when transactions share prefixes.

Results: With identical support and confidence thresholds, both algorithms should return the same mathematically valid frequent itemsets and rules.

Scalability: FP-Growth is generally more suitable for large or dense transactional datasets because it avoids the candidate explosion that can affect Apriori.

11. Discussion

The experiment demonstrates that algorithm choice does not change the definition of a frequent pattern. Instead, it changes how efficiently the search space is explored. Apriori is conceptually simple and useful for teaching the Apriori property, candidate generation and pruning. FP-Growth is more structurally complex but is designed to reduce the cost of candidate generation. On this dataset, both algorithms find the same 126 frequent itemsets because the transaction universe contains only 21 categorical items and the support threshold is moderate. Therefore, the timing difference on this small dataset should not be interpreted as a general benchmark of scalability. On a much larger dataset with thousands of unique items and many long transactions, FP-Growth would generally have a stronger practical advantage.

12. Conclusion

Apriori and FP-Growth were successfully implemented and compared. The preprocessing converted the supplied e-commerce records into a suitable categorical transaction representation. At 5% minimum support, both algorithms identified the same frequent itemsets, and at 50% minimum confidence they produced the same association rules. Apriori is easier to understand and explain, while FP-Growth is generally more efficient because it compresses the transaction database and avoids repeated candidate generation. For educational purposes, implementing both algorithms demonstrates both the logic of frequent-pattern discovery and the importance of selecting an efficient mining strategy.

13. Limitations

The supplied dataset does not contain repeated product purchases per customer, so a conventional product-level market basket could not be constructed from User_ID/Product_ID. The analysis therefore uses categorical attributes within each purchase row. This is appropriate for categorical association mining, but the discovered rules should be interpreted as relationships among purchase attributes rather than statements that two specific products are frequently purchased together.

14. Submission Checklist

1. Dataset: ecommerce_dataset_updated.csv
2. Source code: CIA2_PartA_Association_Rule_Mining.py
3. Frequent itemsets: Apriori and FP-Growth CSV files
4. Association rules: Apriori and FP-Growth CSV files with support, confidence and lift
5. Comparison: algorithm_comparison.csv and this report
6. Verification: verification.csv showing both algorithms agree on frequent itemsets
7. README and requirements files for reproducibility

Note: Execution-time values are machine-dependent. The code automatically measures and reports the actual time on the student's computer. Therefore, use the terminal output from your own VS Code run if the instructor specifically asks for local execution time.